

A HANDS-ON, CODE-FIRST COURSE

LangChain From Zero

Build real GenAI apps with Gemini & local models

PHASE 2 — WORKING WITH MODELS

Prerequisites: Phase 1 done — you can build and run `prompt | model | parser` on both backends.

Targets: Chat models · messages & roles · invoke / stream / batch · temperature & parameters

Estimated time for this phase: 75–100 minutes

Where this phase sits

You can wire Runnables into a chain. Now we zoom in on the most important Runnable of all — the **model** — and learn to talk to it properly: the message format it really wants, the three ways to call it, and the dials that change what it says.

Ph 0 ✓ **Environment Setup**

Ph 1 ✓ **LangChain Fundamentals** — Runnables, LCEL, the  pipe.

Ph 2 **Working with Models** ← you are here
Chat models vs LLMs, messages & roles, invoke / stream / batch, temperature & other parameters.

Ph 3 **Prompts & Output Parsers** — templates, few-shot, structured output with Pydantic.

Ph 4 **Chains & LCEL Composition** — parallel, routing, custom functions, debugging.

Ph 5–8 **Memory · Documents · Embeddings · RAG**

Ph 9–12 **Tools & Agents · LangGraph · Deployment · Capstone**

Phase 2 · Working with Models

WHAT YOU'LL BE ABLE TO DO AFTER THIS PHASE

Explain the difference between a chat model and a legacy LLM; build conversations from System / Human / AI messages; call a model three ways — `invoke` , `stream` , and `batch` — on both Gemini and Ollama; read what comes back (text + token usage); and tune behaviour with `temperature` , `max_tokens` , and friends.

In this phase

1. 2.1 — Chat models vs LLMs (and why we only use chat models)
2. 2.2 — Messages: what a chat model really eats
3. 2.3 — `invoke` : the workhorse, and reading the reply
4. 2.4 — `stream` : token by token
5. 2.5 — `batch` : many inputs at once
6. 2.6 — The dials: temperature, max_tokens & friends
7. 2.7 — Common errors & troubleshooting
8. 2.8 — Mini practice exercise
9. Terms introduced this phase

2.1 — Chat models vs LLMs (and why we only use chat models)

There are two historical flavours of text model. The distinction matters because the old style still lurks in tutorials and will confuse you if you don't know the tell.

	Legacy "LLM" (text completion)	Chat model (what we use)
Input	One plain string	A <i>list of messages</i> , each with a role
Output	A plain string	An <code>AIMessage</code> object (text in <code>.content</code>)
Roles?	No concept of who's speaking	Yes — system / user / assistant
Class name tell	<code>OllamaLLM</code> , <code>GoogleGenerativeAI</code>	<code>Chat Ollama</code> , <code>Chat GoogleGenerativeAI</code>
Status	Legacy / rarely needed	The modern default

THE SIMPLE RULE

Always use the class whose name starts with `Chat` . If a call returns a bare string instead of a message object, you probably grabbed a legacy text-completion LLM by mistake. Chat models won because real applications need *roles* — a way to separate "your standing instructions" from "the user's question" from "what the AI said last turn." That's the next section.

2.2 — Messages: what a chat model really eats

In Phase 1 we handed the model a single string and it worked. That's a convenience — under the hood a chat model always thinks in terms of a **list of messages**, each tagged with a **role**:

Role	Message class	Meaning
system	<code>SystemMessage</code>	Standing instructions / persona. Sets the rules for the whole conversation. Usually first.
user (human)	<code>HumanMessage</code>	What the person says.
assistant (ai)	<code>AIMessage</code>	What the model said on a previous turn. You include these to give it memory of the conversation.
tool	<code>ToolMessage</code>	The result of a tool the model asked to run. (Used from Phase 9 onward.)

There are two equivalent ways to write messages. Create `messages.py` and try the object form:

```
from dotenv import load_dotenv
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_core.messages import SystemMessage, HumanMessage # 1

load_dotenv()
model = ChatGoogleGenerativeAI(model="gemini-2.5-flash")

conversation = [ # 2
    SystemMessage("You are a terse assistant. Answer in one sentence."),
    HumanMessage("What is LangChain?"),
]

response = model.invoke(conversation) # 3
print(response.content) # 4
```

Line	Explanation
1	Import the message classes from <code>langchain_core.messages</code> . (In LangChain 1.x <code>from langchain.messages import ...</code> also works — same classes.)
2	Build the conversation as a list . The <code>SystemMessage</code> sets the rules; the <code>HumanMessage</code> is the actual question.
3	Pass the whole list to <code>invoke</code> . A bare string (Phase 1) is just shorthand for <code>[HumanMessage(...)]</code> with no system message.
4	The reply is an <code>AIMessage</code> ; <code>.content</code> is its text.

Expected output:

LangChain is a framework for building applications powered by large language models.

THE SHORTER "ROLE DICT" FORM

Instead of message objects you can use plain dictionaries — handy and equally valid:

```
conversation = [
    {"role": "system", "content": "You are a terse assistant."},
    {"role": "user", "content": "What is LangChain?"},
]
```

Use whichever you prefer; this course leans on the message *objects* because they're self-documenting and what templates produce in Phase 3.

The Ollama version is the usual one-line swap — import `ChatOllama`, drop `load_dotenv()`, everything else identical:

```
from langchain_ollama import ChatOllama
from langchain_core.messages import SystemMessage, HumanMessage

model = ChatOllama(model="llama3.2")
response = model.invoke([
    SystemMessage("You are a terse assistant. Answer in one sentence."),
    HumanMessage("What is LangChain?"),
])
print(response.content)
```

2.3 — `invoke`: the workhorse, and reading the reply

You met `invoke` already: send input, wait, get one full result. Now let's look properly at *what comes back*. The reply isn't just text — it's an `AIMessage` carrying useful metadata.

```
response = model.invoke("Name three primary colors.")

print(response.content)      # the text: "Red, blue, and yellow."
print(response.text)         # same text via the .text property
print(response.usage_metadata) # token counts for this call
```

Expected (token numbers will vary, and local models may report differently):

```
Red, blue, and yellow.
Red, blue, and yellow.
{'input_tokens': 12, 'output_tokens': 8, 'total_tokens': 20}
```

TERM: USAGE_METADATA

`usage_metadata` tells you how many **tokens** (Phase 0) the call consumed: `input_tokens` (your prompt) + `output_tokens` (the reply). On cloud models this is what you're billed on, so it's worth knowing how to read it. Local Ollama calls are free but still report counts.

2.4 — `stream`: token by token

`invoke` waits for the entire answer before returning. For anything long, that feels slow. `stream` instead hands you the answer in small **chunks** as the model produces them — this is how chat UIs get that "typing" effect.

```
from dotenv import load_dotenv
from langchain_google_genai import ChatGoogleGenerativeAI

load_dotenv()
model = ChatGoogleGenerativeAI(model="gemini-2.5-flash")

for chunk in model.stream("Write a 2-sentence story about a brave teapot."): # 1
    print(chunk.text, end="", flush=True)                                     # 2
print()                                                                    # 3 final
newline
```

Line	Explanation
1	<code>model.stream(...)</code> returns an iterator — a thing you loop over. Each <code>chunk</code> is an <code>AIMessageChunk</code> holding a small piece of the answer.
2	Print each chunk's <code>.text</code> with <code>end=""</code> (no line break between pieces) and <code>flush=True</code> (show it immediately, don't buffer). The result looks like live typing.
3	One newline at the end to tidy the prompt.

Expected output appears *gradually*, ending as:

```
The little teapot stood firm as the kitchen shook. With a defiant whistle, it
poured courage into every trembling cup.
```

RE-ASSEMBLING A FULL MESSAGE FROM CHUNKS

Chunks are designed to be **added together**. If you need the whole message at the end (e.g. to store it), accumulate with `+`:

```
full = None
for chunk in model.stream("..."):
    full = chunk if full is None else full + chunk
print(full.text) # the complete, reassembled answer
```

The Ollama version is identical — just build the model with `ChatOllama(model="llama3.2")`.

2.5 — `batch`: many inputs at once

When you have several *independent* prompts, don't loop and call `invoke` one at a time — that's slow because each waits for the last. `batch` sends them together and runs them in parallel under the hood.

```
questions = [
    "Capital of Japan?",
    "Capital of Brazil?",
    "Capital of Kenya?",
]

responses = model.batch(questions)          # 1 -> a list, same order as input

for q, r in zip(questions, responses):      # 2
    print(f"{q} -> {r.content}")
```

Line	Explanation
1	<code>batch</code> takes a <i>list</i> of inputs and returns a <i>list</i> of <code>AIMessage</code> results, in the same order . Internally these run concurrently, so three questions take about as long as one.
2	<code>zip</code> pairs each question with its answer so we can print them together.

Expected output:

```
Capital of Japan? -> Tokyo
Capital of Brazil? -> Brasília
Capital of Kenya? -> Nairobi
```

TWO USEFUL EXTRAS

- **Results as they finish:** `model.batch_as_completed(inputs)` yields each result the moment it's ready (order not guaranteed; each item carries its input index).
- **Throttle parallelism:** to avoid hammering a rate-limited API, cap concurrency:

```
model.batch(inputs, config={"max_concurrency": 5})
```

Both work on chains too, not just models — because (Phase 1!) everything is a Runnable.

2.6 — The dials: `temperature`, `max_tokens` & friends

You pass these when you *build* the model. They're broadly standard across providers, though each provider supports its own extras.

Parameter	What it does	Typical use
<code>temperature</code>	Randomness/creativity. <code>0</code> = focused & near-repeatable; higher (up to <code>~1-2</code>) = more varied & creative.	<code>0</code> for facts/extraction; <code>0.7-1.0</code> for brainstorming/writing.

<code>max_tokens</code>	Hard cap on the <i>length</i> of the reply (in tokens). The model stops when it hits the limit.	Keep answers short; control cost.
<code>top_p</code>	"Nucleus sampling" — another randomness control. Usually leave at default and tune <code>temperature</code> instead.	Advanced tuning.
<code>timeout</code>	Seconds to wait before giving up on a slow response.	Slow networks / big prompts.
<code>max_retries</code>	How many times LangChain auto-retries on transient errors (rate limits, 5xx). Defaults to <code>6</code> .	Unreliable connections.

See temperature for yourself — ask for something open-ended twice at each setting:

```
from dotenv import load_dotenv
from langchain_google_genai import ChatGoogleGenerativeAI

load_dotenv()
prompt = "Invent a name for a coffee shop. One name only."

cold = ChatGoogleGenerativeAI(model="gemini-2.5-flash", temperature=0)
hot = ChatGoogleGenerativeAI(model="gemini-2.5-flash", temperature=1.2)

print("temp 0 :", cold.invoke(prompt).content, "/", cold.invoke(prompt).content)
print("temp 1.2:", hot.invoke(prompt).content, "/", hot.invoke(prompt).content)
```

Expected pattern (exact names vary): the two cold answers are **identical or near-identical**; the two hot answers **differ**.

```
temp 0 : The Daily Grind / The Daily Grind
temp 1.2: Cloud & Cup / Wandering Bean
```

TERM: TEMPERATURE

Temperature controls how much the model "gambles" when picking each next token. Low temperature → it almost always picks the most likely word (consistent, sometimes dull). High temperature → it more often picks less-likely words (surprising, sometimes incoherent). It is the single dial you'll reach for most.

PARAMETER CAVEATS

Not every provider supports every parameter or the same ranges. Gemini and Ollama both accept `temperature` and `max_tokens`. Some reasoning models ignore or restrict `temperature`. If a parameter is rejected, check that provider's chat-model integration page — but the five above are safe and common.

2.7 — Common errors & troubleshooting

Symptom	Likely cause & fix
---------	--------------------

Reply is a bare string, no <code>.content</code> attribute	You used a legacy text-completion LLM (e.g. <code>OllamaLLM</code> , <code>GoogleGenerativeAI</code>). Switch to the <code>Chat...</code> class.
<code>AttributeError: 'str' object has no attribute 'content'</code>	You added <code>StrOutputParser</code> (or are already on a string) and then asked for <code>.content</code> again. A parsed string <i>is</i> the text — drop the <code>.content</code> .
Streaming prints nothing until the very end	Something downstream is buffering. Ensure you print <code>chunk.text</code> with <code>flush=True</code> , and that no non-streaming step sits in between.
Answer is cut off mid-sentence	<code>max_tokens</code> is too low. Raise it or remove it.
Gemini: <code>temperature</code> error / out of range	Use a value the model accepts (0–2 is safe for <code>gemini-2.5-flash</code>); some models cap lower.
<code>usage_metadata</code> is <code>None</code>	Some local/streaming paths don't populate it. Expected; not an error.

2.8 — Mini practice exercise

YOUR TURN (DO THIS BEFORE PHASE 3)

Task: Write `persona_chat.py` that gives the model a `SystemMessage` persona of your choice (e.g. "a grumpy pirate"), then **streams** its answer to a question. Print the model's `usage_metadata` at the end. Finally, run the same persona + question through **both** Gemini and Ollama and compare the styles.

Why: it combines everything in this phase — message roles, streaming, reading metadata, and backend-swapping — which is exactly the toolkit Phase 3 builds prompts on top of.

Hints (not the full answer):

- To stream *and* capture the full message for its metadata, accumulate chunks with `full = chunk if full is None else full + chunk` (see §2.4), then read `full.usage_metadata` .
- The persona goes in a `SystemMessage` ; the question in a `HumanMessage` ; pass both as a list.
- Swap backends by changing only the line that builds the model.
- If Ollama streaming feels slow on the first run, that's the model loading into memory — run it twice.

Terms introduced this phase

Term	One-line meaning
Chat model	Modern model that takes a message list and returns an <code>AIMessage</code> ; class starts with <code>Chat</code> .
LLM (legacy sense)	Old text-completion model: string in, string out, no roles.
Message / role	A tagged turn in the conversation: system, user, assistant, or tool.

SystemMessage	Standing instructions / persona for the whole conversation.
HumanMessage	What the user says.
AIMessage	What the model replies; text in <code>.content</code> / <code>.text</code> .
AIMessageChunk	A streamed piece of an answer; chunks add together with <code>+</code> .
usage_metadata	Token counts (input/output/total) for a call.
temperature	Randomness dial: low = focused, high = creative.
max_tokens	Cap on reply length, measured in tokens.
batch / stream	Run many inputs in parallel / receive one answer in live pieces.

PHASE 2 COMPLETE

You now know the model inside-out: chat vs legacy, the message/role format it actually consumes, the three call styles (`invoke` , `stream` , `batch`) on both backends, how to read the reply and its token usage, and how `temperature` and `max_tokens` shape output. **Next — Phase 3: Prompts & Output Parsers**, where you stop hard-coding strings and start building reusable prompt templates (including few-shot examples), then force the model to return clean, structured data with Pydantic.